

Лабораторная работа № 6

Тема: Разделение Express-приложения на routes, controllers и services

Цель: Научиться разделять Express-приложение по ответственности, выносить HTTP-обработчики в controllers, бизнес-операции в services и сохранять в routes только описание endpoints.

Практическое задание

Продолжить разработку **Internet Shop API**.

В предыдущих лабораторных работах маршруты товаров находятся в: routes/productRoutes.js

Необходимо выполнить рефакторинг Product-модуля и разделить его на три части:

- productRoutes.js;
- productController.js;
- productService.js.

После рефакторинга URL и поведение API для клиента не должны измениться.

Последовательность выполнения

1. Создайте каталоги controllers и services.

Структура проекта:

```
express-shop/  
├── controllers/  
├── middleware/  
├── routes/  
├── services/  
├── app.js  
├── package.json  
└── package-lock.json
```

2. Создайте services/productService.js.

На данном этапе данные можно хранить в массиве непосредственно в service.

```
let products = [  
  {  
    id: 1,  
    name: "Keyboard",  
    price: 15000,  
    category: "Accessories",  
    inStock: true  
  },  
  {
```

```
    id: 2,  
    name: "Mouse",  
    price: 8000,  
    category: "Accessories",  
    inStock: true  
  },  
  {  
    id: 3,  
    name: "Monitor",  
    price: 85000,  
    category: "Monitors",  
    inStock: false  
  }  
];
```

3. Реализуйте получение всех товаров в service.

```
export function getAllProducts() {  
  return products;  
}
```

Функция не должна получать req или res.

4. Реализуйте поиск товара по id.

```
export function findProductById(id) {  
  return products.find(  
    product => product.id === id  
  );  
}
```

Обратите внимание: service получает уже готовое значение id, а не req.params.

5. Реализуйте создание товара.

```
export function addProduct(data) {  
  const product = {  
    id: products.length + 1,  
    ...data  
  };  
}
```

```
  products.push(product);
```

```
  return product;  
}
```

Service получает обычный JavaScript-объект data.

6. Реализуйте изменение товара.

Создайте функцию:

```
export function updateProductById(id, changes) {  
  const product = products.find(  
    product => product.id === id  
  );  
  
  if (!product) {  
    return null;  
  }  
  
  Object.assign(product, changes);  
  
  return product;  
}
```

7. Реализуйте удаление товара.

Создайте:

```
export function deleteProductById(id) {  
  const index = products.findIndex(  
    product => product.id === id  
  );  
  
  if (index === -1) {  
    return null;  
  }  
  
  const deletedProduct = products[index];  
  
  products.splice(index, 1);  
  
  return deletedProduct;  
}
```

8. Создайте controllers/productController.js.

Импортируйте функции service:

```
import {  
  getAllProducts,  
  findProductById,  
  addProduct,  
  updateProductById,  
  deleteProductById  
} from "../services/productService.js";
```

9. Создайте controller получения всех товаров.

```
export function getProducts(req, res) {  
  const products = getAllProducts();  
  
  res.json(products);  
}
```

Здесь controller:

- получает HTTP-запрос;
- вызывает service;
- отправляет HTTP-ответ.

10. Создайте controller получения одного товара.

```
export function getProductById(req, res) {  
  const id = Number(req.params.id);  
  
  const product = findProductById(id);  
  
  if (!product) {  
    return res.status(404).json({  
      message: "Product not found"  
    });  
  }  
  
  res.json(product);  
}
```

11. Создайте controller добавления товара.

```
export function createProduct(req, res) {  
  const product = addProduct(req.body);  
  
  res.status(201).json(product);  
}
```

Обратите внимание:

req.body

используется в controller.

В service передаётся только:

data

12. Создайте controller изменения товара.

```
export function updateProduct(req, res) {  
  const id = Number(req.params.id);  
  
  const product = updateProductById(  
    id,  
    req.body
```

```

);

if (!product) {
  return res.status(404).json({
    message: "Product not found"
  });
}

res.json(product);
}

```

13. Создайте controller удаления товара.

```

export function deleteProduct(req, res) {
  const id = Number(req.params.id);

  const product = deleteProductById(id);

  if (!product) {
    return res.status(404).json({
      message: "Product not found"
    });
  }

  res.json(product);
}

```

14. Упростите productRoutes.js.

Импортируйте controllers:

```
import express from "express";
```

```

import {
  getProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct
} from "../controllers/productController.js";

```

```
const router = express.Router();
```

Маршруты должны выглядеть компактно:

```

router.get("/", getProducts);
router.get("/:id", getProductById);
router.post("/", createProduct);
router.patch("/:id", updateProduct);

```

```
router.delete("/:id", deleteProduct);
```

```
export default router;
```

15. Проверьте app.js.

В app.js по-прежнему должно использоваться:

```
app.use(express.json());
```

```
app.use(  
  "/api/products",  
  productRoutes  
);
```

app.js не должен напрямую импортировать productController или productService.

16. Проверьте получение всех товаров.

GET /api/products

Должен вернуться массив товаров.

17. Проверьте получение одного товара.

GET /api/products/1

Должен вернуться товар с id = 1.

Проверьте:

GET /api/products/999

Ожидаемый статус:

404 Not Found

18. Проверьте создание товара.

POST /api/products

JSON:

```
{  
  "name": "SSD 1TB",  
  "price": 45000,  
  "category": "Components",  
  "inStock": true  
}
```

Ожидаемый статус:

201 Created

19. Проверьте частичное изменение.

PATCH /api/products/1

JSON:

```
{  
  "price": 13500,
```

```
"inStock": false  
}
```

Проверьте, что изменились только переданные свойства.

20. Проверьте удаление.

DELETE /api/products/2

После удаления:

GET /api/products/2

должен вернуть 404.

21. Проверьте архитектурное правило.

В productService.js не должно быть обращений к:

req

res

req.body

req.params

res.json()

res.status()

Service должен работать только с обычными JavaScript-значениями.

Итоговая структура

```
express-shop/  
├── controllers/  
│   └── productController.js  
├── middleware/  
│   ├── logger.js  
│   └── checkApiKey.js  
├── routes/  
│   ├── productRoutes.js  
│   ├── categoryRoutes.js  
│   └── orderRoutes.js  
├── services/  
│   └── productService.js  
├── app.js  
├── package.json  
└── package-lock.json
```

Требования к результату

Приложение должно:

- содержать отдельные каталоги routes, controllers и services;
- использовать productRoutes.js только для описания endpoints;
- использовать productController.js для работы с req и res;
- использовать productService.js для операций с товарами;
- не передавать req или res в service;
- сохранять существующие URL API;

- поддерживать GET, POST, PATCH и DELETE;
- возвращать 404 для отсутствующего товара;
- возвращать 201 при создании товара;
- запускаться через существующий app.js.